# Interrupt in Operating Systems

Interrupt: Signal to CPU to handle an event

- Temporarily stops current execution
- After handling, CPU resumes previous task
- Enables efficient multitasking

**OS is interrupt driven.**

Interrupts are **essential** for making an OS efficient, responsive, and multitasking.Without interrupts, modern OS like Linux or Windows would not work properly.

# Why Interrupts Are Needed?

Without interrupts, the CPU would have to continuously check (poll) devices to see if they need attention — which wastes CPU time.

Interrupts allow:

- Efficient CPU utilization
- Fast response to events
- Support for multitasking

# Polling (Non-Vectored Interrupt / Polled Interrupt)

In **polling**, the CPU does NOT know which device caused the interrupt.

It must **check each device one by one** to find the source.

Interrupt Occurs
↓
CPU jumps to common ISR
↓
Check Device A?
Check Device B?
Check Device C?
↓
Find requesting device
↓
Service it

# Polling vs Vectored Interrupt

Polling: CPU continuously checks device status

Interrupt: Device notifies CPU when needed

- Polling wastes CPU cycles
- Interrupt improves efficiency

| **Interrupt** | **Polling** |
|---|---|
| CPU responds only when event occurs | CPU continuously checks device |
| Efficient | Wasteful |
| Used in modern OS | Rarely used |

# Polling vs Vectored Interrupt..

Modern OS use **vectored interrupts.**

✔Polling is still used in:

- Embedded systems
- Very small microcontrollers
- Situations where interrupts are disabled

# Types of Interrupts

- Hardware Interrupt (Keyboard, Disk, Timer)
- Software Interrupt (System Call)
- Maskable Interrupt (Can be disabled)
- Non-Maskable Interrupt (Critical errors)

**Hardware Interrupt**
Generated by hardware devices.
Examples:
- Keyboard key press
- Mouse movement
- Disk I/O completion
- Timer interrupt

**Maskable Interrupt**
- Can be disabled (masked) by the CPU.
- Used for normal devices.

**Non-Maskable Interrupt (NMI)**
- Cannot be ignored.
- Used for critical errors (e.g., hardware failure).

**Software Interrupt**
Generated by a program intentionally.
Example:
- System call (**like read(), write()), trap, exception**

# Software vs Hardware Interrupt

| Feature | Software Interrupt | Hardware Interrupt |
|---|---|---|
| Generated by | Program instruction | External device |
| Timing | Synchronous | Asynchronous |
| Example | System call | Keyboard press |
| Control | Intentional | External event |

# Software Interrupt

All system calls are implemented using software interrupts (or similar mechanisms),

but not all software interrupts are system calls.

# Software Interrupt…

User programs cannot directly access:

- Hardware
- Kernel memory
- Privileged instructions

So they use **software interrupt** to:

✔ Switch from user mode → kernel mode
✔ Ask OS to perform protected operations

# Flow of Software Interrupt (System Call Case)

User Program

↓

Executes INT / syscall instruction

↓

CPU switches to Kernel Mode

↓

Looks up IDT entry

↓

Executes Kernel Service Routine

↓

Returns to User Mode

# System Call Case

write(1, "Hello", 5);

internally:

1. Parameters placed in registers
2. syscall instruction executed
3. CPU jumps to kernel handler
4. Kernel performs write
5. Returns to user program

# How IVT Works Internally

The **Interrupt Vector Table (IVT)** is a table in memory that stores the **addresses of Interrupt Service Routines (ISR)**.

➔ Device sends **interrupt**
➔ CPU receives interrupt number
➔ CPU calculates ISR address from table
➔ CPU jumps to ISR and executes it

# Hardware Interrupt Handling Inside CPU

- 1. Device sends interrupt to controller
  - 2. CPU finishes current instruction
  - 3. CPU saves context (PC, registers)
  - 4. Switch to Kernel Mode
  - 5. Execute ISR
  - 6. Restore context and resume

# Interrupt Handling Flow

- Device → Interrupt Controller → CPU
- ↓
- Save Context
- ↓
- Fetch ISR Address
- ↓
- Execute ISR
- ↓
- Restore Context
- ↓
- Resume Program

# Key points

- **Efficient CPU Utilization**:Avoids busy waiting
- **Enables Multitasking:**Enables context switching
- **Handles I/O Devices Efficiently:**Responsive device communication
- **Supports Real-Time Response:**Immediate response
- **Enables System Calls** (Controlled Kernel Access):Controlled Kernel Access
- **Error and Exception Handling:** Detects Faults
- **Power Efficiency:** Let CPU sleep when idle, Wake up only when needed

# I/O Device Management

Suppose you want to print a document.

**When You Click “Print”**

**Step 1: Application Makes System Call**

The program calls something like:

**write() → print service**

This triggers a **system call (software interrupt)**.

CPU switches to **kernel mode**.

# I/O Device Management

## OS Uses Printer Driver

The OS does NOT directly control hardware.

It uses a **printer driver** (a kernel module).

Driver responsibilities:

- Convert document into printer language (e.g., PCL/PostScript)
- Manage communication with printer
- Send data in correct format

# Spooling

Since printers are slow, OS uses **spooling**.

**What is Spooling?**

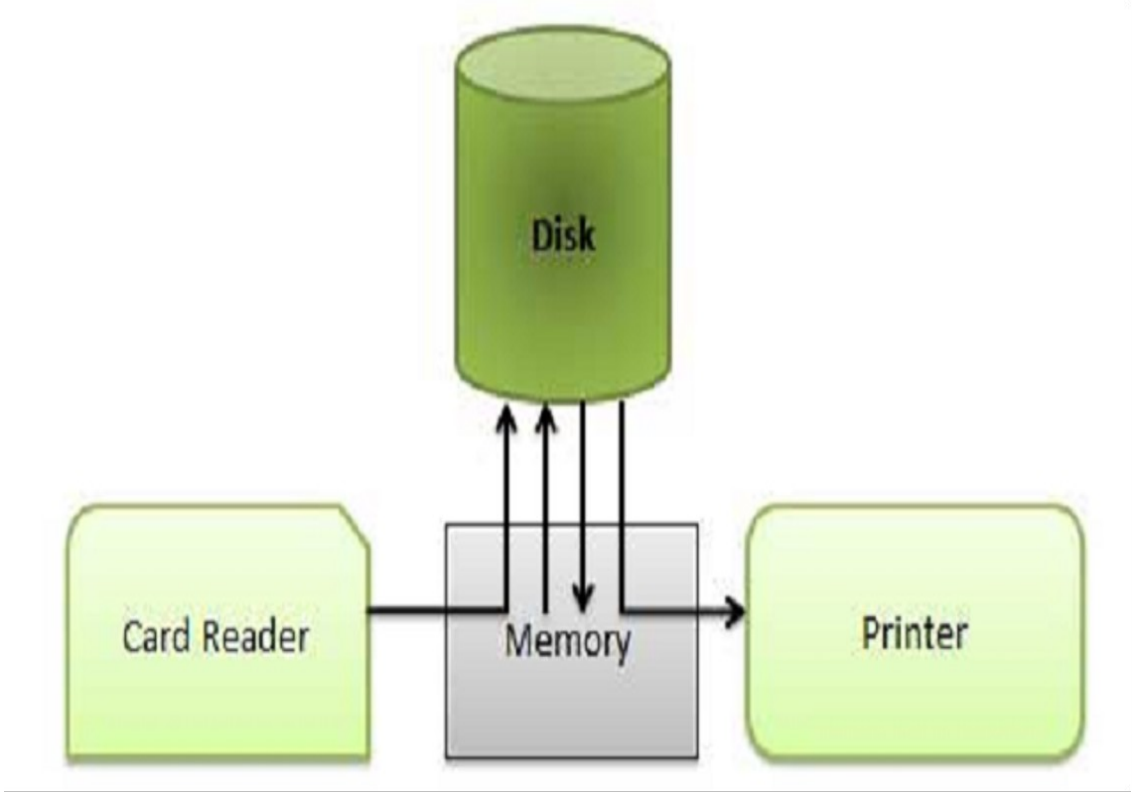
Spooling = Storing print jobs temporarily on disk before sending to printer.

1. OS saves your document in a **print queue**
2. You can continue working
3. OS sends data to printer in background

This prevents:

❌ CPU waiting
❌ Program blocking

# Spooling



# Interrupt-Driven Communication

Printer works using interrupts.

**Process:**

1. OS sends part of data to printer
2. Printer prints it
3. When ready for more data → sends interrupt
4. OS sends next chunk

So CPU is free while printer prints.

# Printer Queue Management

If multiple users print:

- OS maintains a **print queue**
- Jobs are scheduled (usually FIFO)
- OS decides which job prints first

In servers, priority scheduling may be used.

# I/O Device Management..

Application
↓
System Call
↓
Kernel
↓
Printer Driver
↓
Spooler (Print Queue)
↓
Send Data to Printer Buffer
↓
Printer Interrupt
↓
ISR→ Send next data

# What happens without spooling???